COMP34612

Computational Game Theory

Group Project

Adaptive Stackelberg Pricing: From Textbook OLS to 99% Optimality

GROUP 12

A Story of Exploration, Adaptation, and Evidence-Based Iteration

Week 11 | Semester 2 | 2025-26

Problem Overview

Stackelberg Pricing Game Under Imperfect Information

GAME STRUCTURE

Leader sets price u_L, follower responds with u_F

100 days historical data ($t = 1 \ldots 100$)

30 days live gameplay ($t = 101 \ldots 130$)

Follower reaction function is UNKNOWN

Parameters may change over time

WHAT WE KNOW

Demand: $S_L(u_L, u_F) = 100 - 5u_L + 3u_F$

Profit: $(u_L - 1) * S_L$ [unit cost $c_L = 1$]

Objective: maximise 30-day accumulated profit

WHAT WE DON'T KNOW

Follower's payoff function

Follower's strategy space

Whether parameters are stationary

Follower	Strategy Space	Hidden Variant
MK1	[1, +inf)	MK4
MK2	[1, +inf)	MK5
MK3	[1, 15]	MK6

THE CHALLENGE

Learn the follower's reaction function from data, then compute the Stackelberg-optimal price -- all while playing the game live.

L1 Slides 3-5: Stackelberg framework | L4 Slides 3-5: Imperfect information
L7 Slides 6-8: Best reaction functions & equilibrium derivation

The Naive Approach

Why Textbook OLS Alone Scores 85.3%

TEXTBOOK TWO-STEP METHOD (Lecture 4)

Step 1: Fit u_F = a + b*u_L via OLS on historical data (L4 Slide 23)

Step 2: Substitute into profit, solve for optimal u_L (L4 Slide 22)

Step 3: Play u_L* for all 30 days

Result: 85.3% average optimality

Leader	MK1	MK2	MK3	Average
NaiveLeader (textbook OLS)	81.1%	75.4%	99.5%	85.3%
AdaptiveLeader (ours)	99.2%	99.4%	100.0%	99.5%

THE EXTRAPOLATION TRAP

Historical leader prices: u_L in [1.72, 1.90]

Range width: only 0.18 units!

True optimal for MK1/MK2: u_L ~ 20

True optimal for MK3: u_L ~ 11

OLS slope estimated from 0.18-wide range...

...then extrapolated 10x beyond training data

Small slope error = massive price error

OUR FIRST EXPERIMENT

We implemented textbook OLS (L4 Slide 23) and ran it.
Historical OLS gave: beta_hat = 0.42, alpha_hat = 7.1
Predicted optimal: u_L* = 21.3
Actual profit: only 81.1% on MK1 (lost ~5,500)

WHY? The OLS was fit on u_L in [1.72, 1.90].
beta_hat = 0.42 was wrong -- true beta ~ 0.30.
A 0.12 slope error at u_L=20 means a 2.4 price error.

This single experiment defined our entire project direction.

Course Theory Foundation

Building on Lectures 1, 3, 4, 5, 6, and 7

OUR THEORETICAL PIPELINE

L1 & L7: Stackelberg Framework

L1 Slide 3-5: Two-step optimisation: learn R(u_L), then max profit

L7 Slide 6-8: Existence of Stackelberg equilibrium guaranteed

L3: Linear Reaction Function

L3 Slide 12: u_F = alpha + beta*u_L justified by linear demand

L4: OLS for Imperfect Information

L4 Slide 14-20: Learn R_hat(u_L) via least squares regression

L4 Slide 22-23: Two-step approximate Stackelberg strategy

L5: Model Evaluation Metrics

L5 Slide 4-8: RMSE, MAPE, R-squared to assess fit quality

L6: RLS with Forgetting Factor

L6 Slide 9: Weighted LS with forgetting factor lambda

L6 Slide 12: Recursive LS update formula (our implementation)

STACKELBERG OPTIMAL PRICE DERIVATION

Given: u_F = alpha + beta * u_L

Profit = (u_L - 1)(100 - 5u_L + 3(alpha + beta*u_L))

dProfit/du_L = 0 => u_L* = (105 + 3a - 3b) / (10 - 6b)

```
def _optimal_price(self, date=115):
    a = self.alpha + self.gamma * date
    b = self.beta
    denom = 10 - 6 * b
    if denom > 0.1:
        uL = (105 + 3*a - 3*b) / denom
    else:
        # steep slope: ramp up gradually
        uL = prev_uL * 1.5
    return max(1.01, min(uL, self.UPPER_BOUND))
```

leaders.py:69-83 | Direct implementation of Lecture 4 & 7 theory

The Exploration Problem

Why You Can't Just Exploit From Day 101

THE CORE DILEMMA

OLS fitted on u_L in [1.72, 1.90] gives a slope estimate

But the optimal price is u_L ~ 20 (MK1/MK2) or ~11 (MK3)

Plugging a wrong slope into the formula gives wrong u_L*

EXPLORATION-EXPLOITATION TRADE-OFF

Explore: sacrifice 1 day of profit to learn the true slope

Exploit: use learned slope for remaining 29 days

Key question: WHERE to probe?

OUR ITERATIVE DISCOVERY

v1: "Let's probe at 5" -- safe, conservative. Got 98.2%.

We asked: why not higher? Ran v2 with probe at 10: 99.3%

Pushed to 12 (v4) -- still within MK3's [1,15] bound: 99.5%

Each experiment took 1 hour, tested all 3 followers

Probe Price	MK1	MK2	MK3	Avg
u_L = 5 (v1)	97.8%	98.1%	98.9%	98.2%
u_L = 10 (v2)	99.1%	99.2%	99.7%	99.3%
u_L = 12 (v4)	99.2%	99.4%	100.0%	99.5%

EXPLORATION COST-BENEFIT ANALYSIS

Probe day cost: ~500 profit lost vs exploitation
Information gained: true slope (beta) with 10x range
Remaining exploitation: 29 days at near-optimal price
Net gain: ~4,000 profit over naive OLS approach

Feldbaum (1965): "A good controller must simultaneously control AND probe the system."
Our probe embodies this dual control principle.

Ref: Feldbaum, "Dual Control Theory" (1960s)
Ref: Bar-Shalom & Tse, "Dual Effect" (1974)

Our Innovation: Adaptive Exploration

Probe, Learn, Converge in 2 Days

DECISION PIPELINE

1. HISTORICAL PHASE (Days 1-100)

Load 100 days of [u_L, u_F] data

MAD-based outlier filtering (10x threshold)

Detect time trends (correlation > 0.7)

Fit OLS: u_F = alpha + beta*u_L [+ gamma*t]

2. EXPLORATION (Day 101)

Probe at u_L = 12 (or 15 for bounded followers)

Single observation far outside historical range

3. EXPLOITATION (Days 102-130)

Batch OLS refit every 3 days (with new data)

RLS updates between refits (forgetting factor 0.97)

Compute Stackelberg-optimal price each day

```
def new_price(self, date):
if date > 101:
    prev_uL, prev_uF = self.get_price_from_date(date - 1)
    self.all_uL.append(prev_uL)
    self.all_uF.append(prev_uF)
    if date <= 105 or (date - 101) % 3 == 0:
        self._fit_ols()      # batch refit
    else:
        self._rls_update(prev_uL, prev_uF)  # incremental
if date == 101:
    return min(12.0, self.UPPER_BOUND)  # EXPLORE
return self._optimal_price(date)        # EXPLOIT
```

HYBRID OLS+RLS RATIONALE

OLS batch refit: corrects accumulated drift
RLS between refits: responds to daily changes
Best of both: stability + adaptivity

leaders.py:95-107 | Core decision logic (13 lines)

Time Trend Detection

Automatically Adapting to Non-Stationary Followers

HOW WE DISCOVERED THIS (v3)

After v2 scored 99.3%, we noticed MK2 was our weakest: 97.1%

We plotted MK2's u_F over time -- clear upward drift

R-squared on standard model was only 0.91 for MK2

Residuals showed systematic time pattern

EXPERIMENT: ADD TIME REGRESSOR

Extended model: u_F = alpha + beta*u_L + gamma*t

Pearson correlation test: if |corr(t, u_F)| > 0.7, add gamma*t

Result: MK2 jumped from 97.1% to 99.4% (+2.3%)

MK1/MK3 unchanged (correctly detected as stationary)

DECISION

Average score didn't change (99.3% -> 99.3%)

We almost removed it -- but MK2 improvement was critical

Lesson: average metrics can hide per-follower improvements

L4 Example Class Q3: "if linear poor, try extended model"

```
def _detect_time_trend(self):
    corr = np.corrcoef(
        np.arange(1, 101),
        self.hist_uF
    )[0, 1]
    if abs(corr) > 0.7:
        self.use_time = True

def _fit_ols(self):
    X = np.column_stack([np.ones_like(uL), uL])
    if self.use_time:
        X = np.column_stack([X, np.array(self.all_dates)])
    theta = np.linalg.lstsq(X, uF, rcond=None)[0]
    self.alpha, self.beta = theta[0], theta[1]
    self.gamma = theta[2] if self.use_time else 0.0
```

Scenario	Without Time	With Time
MK2 (trending)	97.1%	99.4%
MK1 (stationary)	99.2%	99.2%
MK3 (stationary)	100.0%	100.0%

Online Learning: OLS + RLS Hybrid

Lecture 6 Theory in Practice

THE EXPERIMENT SEQUENCE

Experiment 1: Pure OLS (batch refit every 5 days)

Result: 98.8%. On day 107 we noticed the price was stale -- the model hadn't updated in 5 days, missed a parameter shift.

Experiment 2: Pure RLS (update every day)

Result: 98.8%. RLS drifted -- accumulated small errors without a global correction. Variance across runs was higher.

Experiment 3: OLS+RLS Hybrid (our approach)

Batch OLS refit days 102-105, then every 3 days

RLS with lambda=0.97 (L6 Slide 9) between batch refits

Result: 99.0% avg -- best of both worlds

Experiment 4: Lambda tuning (3 values tested)

lambda=0.95: too aggressive, profit unstable day-to-day

lambda=0.97: stable convergence, chosen for final

lambda=0.99: retained stale history, MK1 variance 245->390

```
def _rls_update(self, uL, uF):
x = np.array([1.0, uL])
P2 = self.P[:2, :2]
Px = P2 @ x
gain = Px / (self.lam + x @ Px)
theta = np.array([self.alpha, self.beta])
theta += gain * (uF - (self.alpha + self.beta * uL))
self.alpha, self.beta = theta[0], theta[1]
self.P = (P2 - np.outer(gain, x @ P2)) / self.lam
```

LECTURE 6 CONNECTION

RLS update formula:
theta_new = theta_old + L * (y - phi'*theta_old)

L = P*phi / (lambda + phi'*P*phi)
P_new = (P - L*phi'*P) / lambda

Our implementation follows this exactly
(leaders.py:85-93)

What Didn't Work

Honest Assessment of Failed Experiments

EXPERIMENT: LAMBDA TUNING (v2-v4)

Hypothesis: higher lambda retains more history for stability

lambda=0.99: MK1 profit variance jumped 245 -> 390

Root cause: old data from [1.72,1.90] "contaminated" post-probe estimates

Decision: lambda=0.97 gave the best variance-bias trade-off

EXPERIMENT: 2-DAY EXPLORATION (v2)

Hypothesis: 2 probe days give more precise slope estimate

Result: 98.9% (worse than 1-day probe at 99.3%)

Root cause: lost 1 exploitation day (~1000 profit) for marginal precision

Decision: single-day probe is optimal cost-benefit

EXPERIMENT: THOMPSON SAMPLING (v4)

Hypothesis: Bayesian exploration would find better probes

We spent 3 days implementing posterior sampling over beta

Result: 99.5% -- identical to deterministic approach

Why: one-shot probe eliminates the sequential E-E tradeoff

Decision: Occam's Razor -- simpler deterministic approach kept

Experiment	Version	Result	vs Final
lambda=0.99	v2 test	Higher var	Worse
lambda=0.95	v2 test	Unstable	Worse
OLS-only	v1	98.8%	-0.2%
2-day probe	v2 test	98.9%	-0.6%
Thompson S.	v4 test	99.5%	+0.0%
Polynomial fit	v3 test	99.3%	+0.0%

REFLECTION

Our biggest mistake was spending 3 days on Thompson Sampling before running a simple test.

If we started again: prototype quickly with a 1-hour experiment before committing days to full implementation.

Every experiment above taught us something that shaped the final 124-line design.

Robustness & Stress Testing

18 Adversarial Scenarios, 17/18 Score >= 99%

Scenario Category	Tests	Avg Score
Standard (MK1-3)	3	99.5%
Negative slopes	2	100.0%
Different trends	3	99.1-99.7%
Nonlinear reactions	3	99.9-100%
High noise (sigma=5)	2	99.5%
Mixed parameters	3	99.3%
Steep slope (beta=2)	1	97.2% -> fixed
Edge cases	1	99.8%

HANDLING MK3/MK6 (BOUNDED)

Strategy space [1, 15] requires capped leader

BoundedAdaptiveLeader: identical logic, UPPER_BOUND = 15

Probe at min(12, 15) = 12 (within bounds)

MK3 result: 100.0% optimality

BUG FOUND: STEEP SLOPE (v7)

Stress test with beta=2.0: price jumped to infinity on day 103!

Root cause: denom = 10-6*2 = -2, formula gave negative u_L

Fix: gradual ramp-up (u_L *= 1.5/day) when denom < 0.1

HIDDEN FOLLOWERS (MK4, MK5, MK6)

These are "slightly modified" versions of MK1-3. Our adaptive approach handles them automatically because we learn the reaction function from scratch for each follower. No hard-coded parameters. The same algorithm generalises to unseen variants.

Development Journey

7 Iterations from 98.2% to 99.6%

Version	Change	Score	What Surprised Us
v1	OLS+RLS, probe u_L=5	98.2%	Probe too conservative -- 5 not far enough
v2	Probe at 10, refit/5 days	99.3%	Biggest single improvement: +1.1%!
v3	Time-trend detection	99.3%	Avg unchanged but MK2 jumped +2.3%
v4	Probe at 12, refit/3 days	99.5%	Diminishing returns on probe distance
v5	Outlier filter, metrics	99.5%	Needed for robustness, not performance
v6	Mixin for GUI dropdown	99.6%	Colab required specific class hierarchy
v7	Steep slope ramp-up	99.6%	Found via stress test -- formula broke!

ABLATION STUDY

What each component adds:

Exploration probe

+14.2% (85.3% -> 99.5%)

OLS+RLS hybrid

+0.2% vs OLS-only

Time trend detection

+2.3% on trending followers

Outlier filtering

Prevents corrupted data

Bounded variant

Handles [1,15] constraint

METHODOLOGY: Every version was tested against all 3 followers in both TEST and MARK (20-run) modes. We only kept changes that improved or maintained performance. v3 nearly got removed because avg was +0.0% -- individual follower testing saved it. Total: 7 versions, 6 failed experiments, 124 lines of final code.

Final Results

Performance Against All Followers

Follower	Profit (TEST)	% of Benchmark	MARK Mode (20 runs)	Benchmark
MK1	28,841	99.2%	98.8% +/- 1.1%	29,061
MK2	31,571	99.4%	99.6% +/- 0.1%	31,748
MK3	16,030	100.0%	98.5% +/- 0.5%	16,034
Average	--	99.5%	99.0%	--

IMPLEMENTATION

Total: 124 lines of Python
Dependencies: numpy only
Two leader classes:
AdaptiveLeader (unbounded)
BoundedAdaptiveLeader (capped)
Shared _AdaptiveCore mixin

MODEL QUALITY METRICS (Lecture 5)

Metric	MK1	MK2	MK3
RMSE	0.12	0.18	0.05
MAPE	1.8%	2.3%	0.9%
R-squared	0.994	0.991	0.998

vs NAIVE LEADER

	Adaptive	Naive	Improvement
MK1	99.2%	81.1%	+18.1%
MK2	99.4%	75.4%	+24.0%
MK3	100.0%	99.5%	+0.5%
Avg	99.5%	85.3%	+14.2%

External Literature & Research

Connecting Our Approach to Published Work

PAPERS AND REFERENCES

Feldbaum, A.A. (1965) Optimal Control Systems

Dual Control: controller must control AND probe simultaneously

Our Day 101 probe = "cautious probing" -- sacrifice one day

Days 102+: certainty-equivalence exploitation

Bar-Shalom & Tse (1974) IEEE TAC

"Dual Effect, Certainty Equivalence and Separation"

CE principle: treat OLS estimates as ground truth after probing

We apply CE from Day 102: use estimated slope as if exact

Letchford et al. (2009) AAMAS

"Learning and Approximation in Stackelberg Games"

Proves polynomial sample complexity for learning reactions

Validates: OLS is theoretically sufficient for this task

Balcan et al. (2015) NeurIPS

"Commitment Without Regret: Online Stackelberg"

Shows diminishing exploration is optimal in repeated games

Our 1-day probe then full exploitation mirrors this insight

Thompson, W. R. (1933) Biometrika

"On the Likelihood That One Unknown Probability..."

We tested Thompson Sampling: 99.5% (identical to deterministic)

Why: one-shot probe eliminates the sequential E-E tradeoff

Occam's Razor: chose simpler deterministic approach

Course Material Integration

Lecture 4 Slide 23: Two-step approximate Stackelberg

Lecture 6 Slide 12: RLS recursive updating formula

Lecture 6 Slide 9: Weighted LS with forgetting factor

Lecture 5: RMSE, MAPE, R-squared evaluation

Key Lessons Learned

What We'd Tell Groups Starting This Project

EXPLORATION > EXPLOITATION

The single biggest improvement (+14.2%) came from one probe day

Historical data is dangerously narrow: [1.72, 1.90]

Without exploration, you're building on sand

Cost: 1 day of suboptimal profit

Reward: 29 days of near-optimal pricing

ROBUSTNESS > PRECISION

99.0% across 20 random seeds > 99.5% on one run

Outlier filtering prevents data corruption

Safety bounds prevent negative demand

Steep slope ramp-up handles edge cases

SIMPLICITY WINS

124 lines of code beat complex approaches

Thompson Sampling = same result, more complexity

Each feature must justify its existence empirically

numpy-only: no external dependencies

TEST EVERYTHING

Every version tested against all 3 followers

20-run MARK mode for statistical significance

18 stress scenarios for edge cases

Ablation study: isolate each component's contribution

IF WE DID THIS AGAIN...

Prototype experiments in 1 hour before committing days

Test per-follower scores, not just averages (v3 lesson)

Run stress tests from day 1, not after v5

Explore multi-probe strategies (days 101+106)

Try nonlinear reaction models earlier (L4 suggests this)

LIMITATIONS

Assumes linear reaction functions

Single probe day -- could miss complex dynamics

lambda=0.97 tuned on MK1-3; MK4-6 may differ

No closed-form regret bound (empirical only)

Conclusion

WHAT WE BUILT

An adaptive Stackelberg pricing leader that achieves 99.5% optimality

Integrates OLS, RLS, and Stackelberg theory from Lectures 1-7

Goes beyond textbook methods with adaptive exploration

Validated through extensive empirical testing

Robust across 18 adversarial scenarios

KEY INNOVATION

Single-day adaptive exploration that discovers the true reaction function slope, enabling convergence to near-optimal pricing within 2 days.

KEY NUMBERS

99.5%	average optimality (TEST mode)
99.0%	average optimality (20-run MARK mode)
+14.2%	improvement over naive OLS
124	lines of Python code
7	development iterations
18	stress test scenarios passed
5	external papers referenced
6	lectures directly integrated

"The gap between textbook theory and optimal practice is not in the math -- it's in the data."

Code: game/src/leaders.py | Group 12 | COMP34612

ANY QUESTIONS?